IFN647 Week 2 Review Questions
Solutions

Question 1 (Text statistic)

Please open a text or an XML file (e.g., "*6146.xml"*) and represent it as a list of paragraphs or sentences, *text*. You may remove any non-relevant information (e.g., '<p>', '</p>', '\n'). After that, you need to find all terms and their frequencies (the number of occurrences in the file) if their length > 2, represent them using a dictionary, *doc*; and print the number of total terms (e.g., 137). Then print the top-10 terms in *doc* in a descending order, e.g.,

[('the', 8), ('technical', 2), ('bounce', 2), ('said', 2), ('and', 2), ('not', 2), ('due', 2), ('rose', 2), ("argentina's", 2), ('argentine', 1)]

At last, please plot the distribution of the top-10 terms (see, Fig 1 for an example).

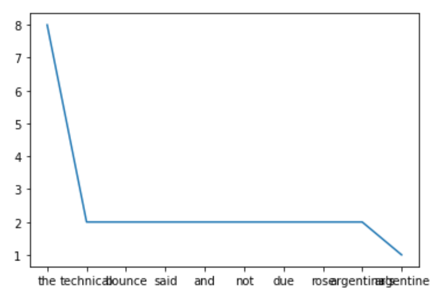


Fig 1. Example of distribution of top-k frequent words.

Solution:

```
myfile=open('6146.xml')
file_=myfile.readlines()
```

```
doc={}
word_count=0
```

```
text = [line.replace("<p>", "").replace("</p>", "").replace("\n", "") for line in file_ if line.startswith('<p>')]
print(text)
```

```
['Argentine bonds were slightly higher in a small technical bounce Wednesday amid low volume.', 'A trader at a large
foreign bank said there was a slight technical bounce at the opening, and he did not expect prices to change much dur
ing the session as no market-moving news is expected.', "The 5.5 percent dollar-denominated Bocon Previsional 2 due 2
001 rose $0.15 to 115.15. Argentina's FRB due 2005 rose 1/8 to 77-3/8.", "&quot;There is general uncertainty,&quot; s
aid the trader, pointing to all the events the market is waiting for, including the passage of the government's new e
conomic measures through Congress, which is now not expected until early October.", "In addition, traders are awaitin
g a meeting Friday between Economy Minister Roque Fernandez and an International Monetary Fund delegation on Argentin
a's fiscal deficit.", '-- Axel Bugge, Buenos Aires Newsroom, 541 318-0668']
```

```
for line in text:
    for term in line.split():
        word_count += 1
        term = term.lower()
        if len(term) > 2:
            try:
                doc[term] += 1
            except KeyError:
                doc[term] = 1
print(word_count)
```

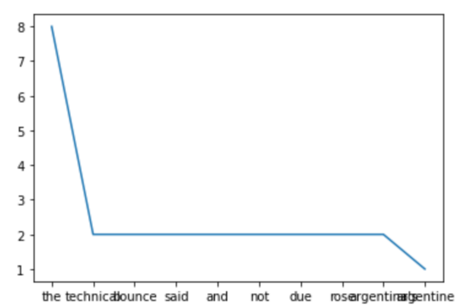
```
137
```

Sort the dictonary and plt the frequency distribution

```
import matplotlib.pyplot as plt
myList = doc.items()
myList = sorted(myList, key=lambda x:x[1], reverse=True) #sorted by value
myLList = list(myList)
myList = myLList[:10]
print(myList)
x, y = zip(*myList)

plt.plot(x, y)
plt.show()
```

```
[('the', 8), ('technical', 2), ('bounce', 2), ('said', 2), ('and', 2), ('not', 2), ('due', 2), ('rose', 2), ("argenti
na's", 2), ('argentine', 1)]
```



Question 2 Which of the following is FALSE? and explain why it is FALSE.

(1) Stemming is a component of text processing that captures the relationships between different variations of a word.
(2) Stemming reduces the different forms of a word that occur because of inflection (e.g., plurals, tenses) or derivation (e.g., making a verb into a noun by adding the suffixation) to a common stem.
(3) In general, using a stemmer for search applications with English text produces a small but noticeable improvement in the quality of results.
(4) A dictionary-based stemmer uses a small program to decide whether two words are related, usually based on knowledge of word suffixes for a particular language.

Solution: (4)
A dictionary-based stemmer has no logic of its own, but instead relies on pre-created dictionaries of related terms to store term relationships. In contrast, an algorithmic stemmer uses a small program to decide whether two words are related, usually based on knowledge of word suffixes for a particular language.

Question 3 (N-grams)

Typically, *n*-grams are formed from overlapping sequences of words., i.e. move *n*-word "window" one word at a time in a document. For example, *bigrams are* 2 words sequences, and *trigrams* are 3 words sequences.

The definition of Tropical fish is described in the following document:

> **Tropical** **fish** are generally those **fish** found in aquatic **tropical** environments around the world, including both **freshwater** and **saltwater** species. Fishkeepers often keep **tropical** **fish** in **freshwater** and **saltwater** **aquariums**.

Please design a python program to print all bigrams and trigrams of the above document that contain at least one of the highlighted key words ('fish', 'tropical', 'freshwater', 'saltwater', 'aquariums').

Solution:

```
import string

def is_highlight(gram, keywords):
    x = False
    for y in keywords:
        if y in gram:
            x = True
    return x

line ="Tropical fish are generally those fish found in aquatic tropical environments around the world, \
including both freshwater and saltwater species. Fishkeepers often keep tropical fish in freshwater and \
saltwater aquariums."

line = line.translate(str.maketrans('','', string.digits)).translate(str.maketrans(string.punctuation, \
                                                                     ' '*len(string.punctuation)))
words=line.split()
stems = words
bigrams = [stems[i]+' '+stems[i+1] for i in range(len(stems)-1)]
trigrams = [stems[i]+' '+stems[i+1]+' '+stems[i+2] for i in range(len(stems)-2)]

keywords = ['fish', 'tropical', 'freshwater', 'saltwater', 'aquariums']

bigrams1 = [gram for gram in bigrams if is_highlight(gram, keywords)]
trigrams1 = [gram for gram in trigrams if is_highlight(gram, keywords)]
print(bigrams1)
print(trigrams1)
```

```
['Tropical fish', 'fish are', 'those fish', 'fish found', 'aquatic tropical', 'tropical environments', 'both freshwat
er', 'freshwater and', 'and saltwater', 'saltwater species', 'keep tropical', 'tropical fish', 'fish in', 'in freshwa
ter', 'freshwater and', 'and saltwater', 'saltwater aquariums']
['Tropical fish are', 'fish are generally', 'generally those fish', 'those fish found', 'fish found in', 'in aquatic
tropical', 'aquatic tropical environments', 'tropical environments around', 'including both freshwater', 'both freshw
ater and', 'freshwater and saltwater', 'and saltwater species', 'saltwater species Fishkeepers', 'often keep tropical
', 'keep tropical fish', 'tropical fish in', 'fish in freshwater', 'in freshwater and', 'freshwater and saltwater', '
and saltwater aquariums']
```

Question 4 (Html tags)

(a) Design a python program to extract all hyperlinks (or destination links) in a html file. You can use HTMLParser python package. You may download a real html file (e.g., "html_example.html") to test it.

(b) What ethical issues might exist in extracting hyperlinks from the Web?

Solution:

(a)

```
from html.parser import HTMLParser
class Parser(HTMLParser):
  # method to append the start tag to the list start_tags.
  def handle_starttag(self, tag, attrs):
    global start_tags
    global attrs_names
    if tag=='a':
        start_tags.append(tag)
        attrs_names.append(attrs)
start_tags = []
attrs_names = []
# Creating an instance of our class.
parser = Parser()

# open a html file
myHtml=open('html_example.html')
file_html=myHtml.read()

# Poviding the input.
parser.feed(file_html)
#print("start tags:", start_tags)
#print("attributes:", attrs_names)
hyper_links = [x[0][1] for x in attrs_names]
print(hyper_links)
myHtml.close()
```

```
['/wiki/Fish', '/wiki/Tropics', '/wiki/Fresh_water', '/wiki/Sea_water', '/wiki/Fishkeeping', '/wiki/List_of_marine_aq
uarium_fish_species', '/wiki/Aquarium', '/wiki/Iridescence', '/wiki/Pigment']
```

(b)

When you design a program (or web crawler) to extract hyperlinks, it possibly involves querying a website repeatedly and accessing a potentially large number of pages. We know that for each page, a request is sent to the Web server, and the server will process the request and send a response back to the computer that is running your code. Each request will consume resources on the server, during which it will not have time to do something else. If your code sends too many such requests over a short span of time, the code can prevent other "normal" users from accessing the site. So, the possible ethical issue looks like your code is a Denial of Service (DoS) attack.

It is also important to make sure that the extraction is legal. If the terms and conditions of the website you are using forbid downloading and copying its content, then this is a possible ethical issue for extracting its content.

PS: If your project is concerned about the legal implications of using web crawler, it is best to seek advice from a professional who has knowledge of the intellectual property (copyright) legislation.